This is a regex in Go, what is it doing?
"^(([a-zA-Z0-9]|[a-zA-Z0-9][a-zA-Z0-9-]*[a-zA-Z0-9]).)*([A-Za-z0-9]|[A-Za-z0-9][A-Za-z0-9-]*[A-Za-z0-9])$"

This regular expression (regex) is used to match a string that adheres to the format of a DNS hostname (not including the trailing dot), which may also include subdomains. Here's a more detailed breakdown:

- The `^` character marks the start of the line.
- `(([a-zA-Z0-9]|[a-zA-Z0-9][a-zA-Z0-9-]*[a-zA-Z0-9]).)*` : This is the part of the regex that matches any subdomains. It's made up of a couple of parts:
  - `[a-zA-Z0-9]` matches any single alphanumeric character.
  - `[a-zA-Z0-9][a-zA-Z0-9-]*[a-zA-Z0-9]` matches an alphanumeric character, followed by any number of alphanumeric characters or dashes, and ends with an alphanumeric character. This pattern ensures that if a dash is used, it must be surrounded by alphanumeric characters (which is a requirement in domain names).
  - The `.` at the end of this section is there to match the dot that separates subdomains.
  - The outermost `()` encloses this whole pattern, and the `*` at the end allows for zero or more of this pattern.
- `([A-Za-z0-9]|[A-Za-z0-9][A-Za-z0-9-]*[A-Za-z0-9])` : This part matches the actual domain name (which follows the same pattern as the subdomains).
- The `$` character marks the end of the line.

So, this regular expression is matching DNS hostnames like "example.com", "sub.example.com", or even "a.b.c.example.com". It doesn't allow for trailing dots, leading or trailing dashes, or multiple consecutive dots.

Is this the correct way to validate a hostname?

The regex pattern you provided is a decent way to validate a large subset of hostnames, but it's not perfect. There are edge cases and specifics that it does not cover. Here are a couple of them:

1. **Length**: The total length of a full domain name (including dots) is limited to 253 characters, and individual labels (each part between the dots) are limited to 63 characters. The provided regex does not account for these restrictions.
2. **International Domain Names (IDNs)**: These domain names can include characters outside the traditional a-z, 0-9, and hyphen. Your regex does not account for these characters. IDNs are converted to Punycode for DNS lookups, so you could argue that you should only need to validate the Punycode version, but that depends on your specific application.
3. **Underscores**: While underscores (_) are not permitted in hostnames according to the specification, some systems do use them and they are often permitted in practice. If you need to allow for these, your regex would need to change.

So, to fully validate a hostname, you'd need to take these things into account, and possibly others depending on your exact use case. However, if you're sure that your hostnames will always be fairly "normal", this regex could be fine for you.

In any case, it's important to remember that validation by regex only provides a preliminary check. If you're doing something like resolving the hostname to an IP address, the lookup itself will be the ultimate test of whether the hostname is valid or not.

Show me the correct regex then

As mentioned, crafting a regex that accurately matches all valid hostnames including Internationalized Domain Names (IDNs) and accounting for length restrictions can be complex and still would not cover all cases (like Punycode representation of IDNs).

But if you're interested in a regex that caters to the standard a-z, 0-9, and hyphen characters in domain names and labels with length restrictions, then you can try the following:

regex Copy code

```
^(?:(?:(?:[a-zA-Z0-9](?:[a-zA-Z0-9\-]{0,61}[a-zA-Z0-9])?)\.){1,126}(?:[a-z
```

This regex matches the domain name following these rules:

1. Each label (the part between dots) must start and end with an alphanumeric character.
2. The middle of each label can contain dashes, but no more than 61 of them.
3. The total number of labels (including the top-level domain but not including a trailing dot) is limited to 126.
4. Each label (including the top-level domain) is limited to 63 characters.

Please remember that this regex doesn't cover edge cases like IDNs or Punycode. It would be best to use a well-tested library specific to your programming language to handle the complexity of hostname validation, if possible.